BINARY SEARCH AND AVL TREES



BINARY SEARCH TREE

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The value of the key of the left sub-tree is less than the value of its parent (root) node's key.
- The value of the key of the right sub-tree is greater than or equal to the value of its parent (root) node's key.

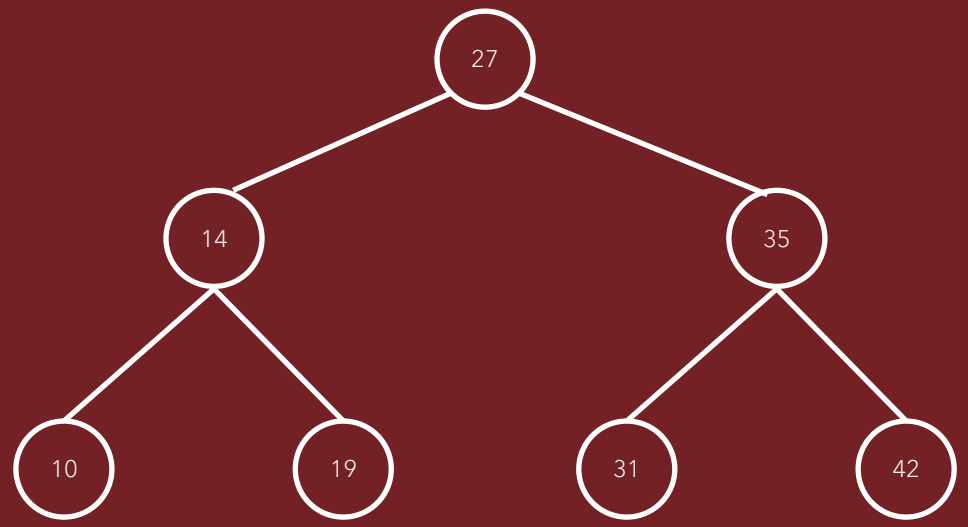
Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right subtree and can be defined as:

left subtree (keys) < node (key) ≤ right subtree (keys)

BINARY SEARCH TREE

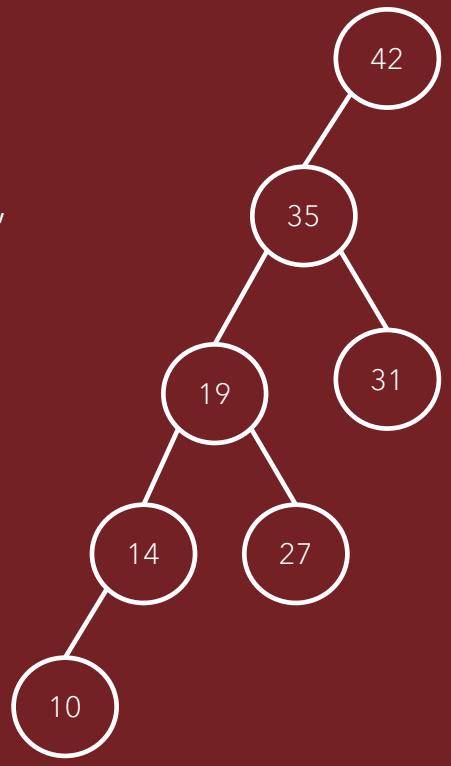
Basic operations of a tree.

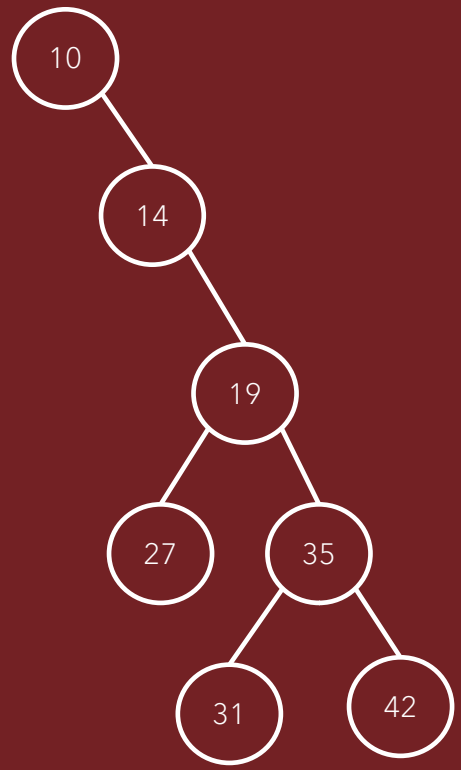
- Search - Searches an element in a tree.
- Insert – Inserts an element in a tree.
- Pre-order Traversal – Traverses a tree in a pre-order manner.
- In-order Traversal – Traverses a tree in an in-order manner.
- Post-order Traversal – Traverses a tree in a post-order manner.



What if the input to binary search tree comes in a sorted (ascending or descending) manner?

If input ‘appears’ Non-increasing manner



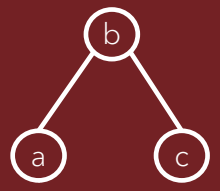


If input ‘appears’ non-decreasing manner

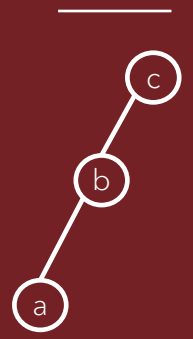
AVL TREES

AVL Trees was named after their inventors Adelson, Velski, and Landis, AVL Trees are height balancing binary search tree. AVL Tree checks the height of the left and right sub-trees and assures that the difference is not more than 1. This difference is called the Balance Factor.

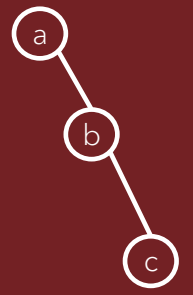
Here we see that the first tree is balanced and the next two trees are not balanced:



Balanced



Not Balanced



Not Balanced

In the second tree, the left subtree of C has height 2 and the right subtree has height 0, so the difference is 2. In the third tree, the right subtree of A has height 2 and the left is missing, so it is 0, and the difference is 2 again. AVL tree permits difference (balance factor) to be only 1.

BalanceFactor = height(left-subtree) − height(right-subtree)

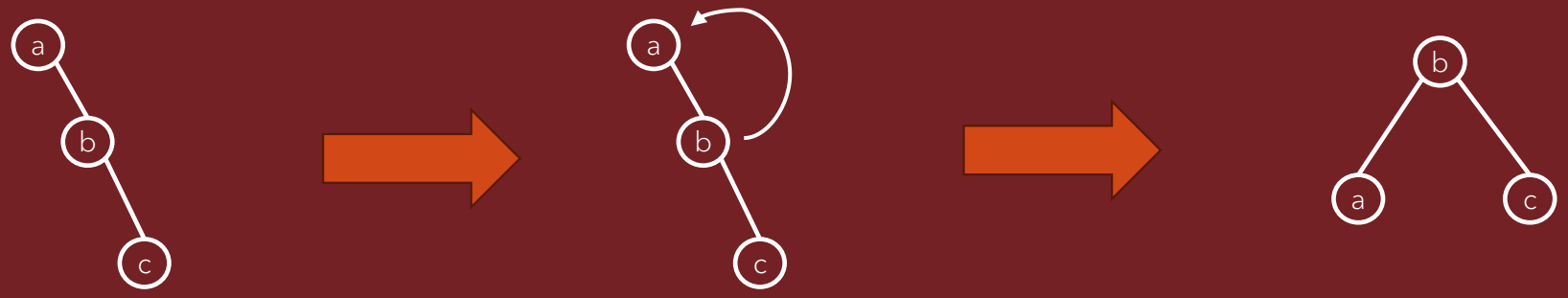
If the difference in the height of left and right sub-trees is more than 1, the tree is balanced using some rotation techniques.

AVL ROTATIONS

- Left Rotation
- Right Rotation
- Left-Right Rotation
- Right-Left Rotation

LEFT ROTATION

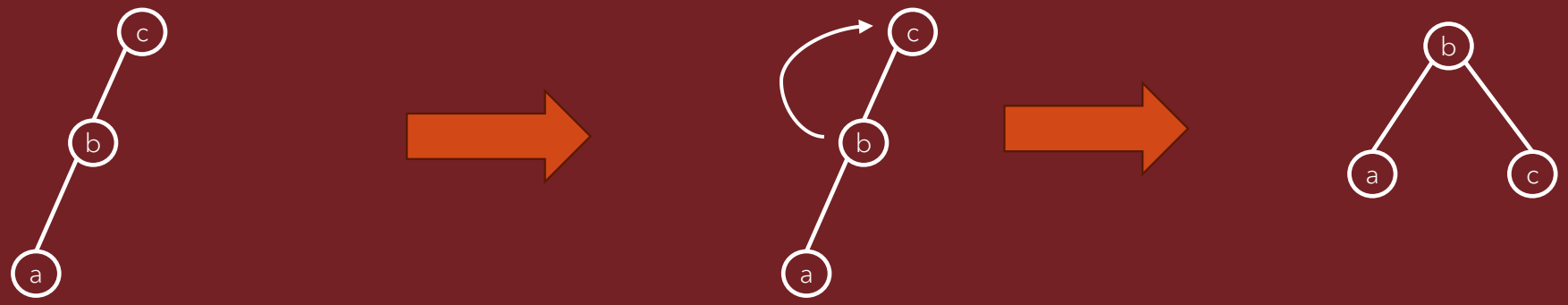
If a tree becomes unbalanced, when a node is inserted into the right subtree of the right subtree, then we perform a single left rotation.



In our example, node A has become unbalanced as a node is inserted in the right subtree of A's right subtree. We perform the left rotation by making A the left-subtree of B.

RIGHT ROTATION

AVL tree may become unbalanced, if a node is inserted in the left subtree of the left subtree. The tree then needs a right rotation.



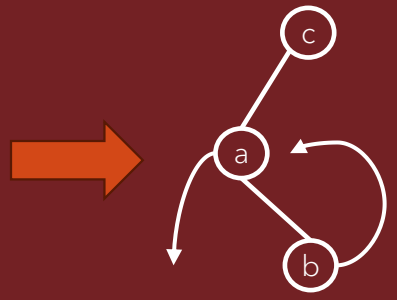
As depicted, the unbalanced node becomes the right child of its left child by performing a right rotation.

LEFT-RIGHT ROTATION

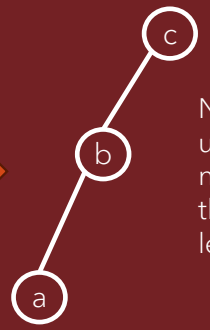
- A **left-right rotation** is a combination of left rotation followed by right rotation.



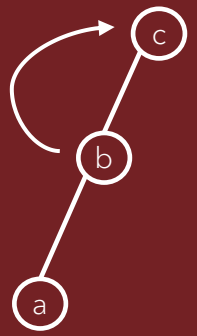
A node has been inserted into the right subtree of the left subtree. This makes C an unbalanced node. These scenarios cause AVL tree to perform left-right rotation.



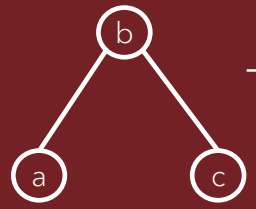
We first perform the left rotation on the left subtree of C. This makes A, the left subtree of B.



Node C is still unbalanced, however now, it is because of the left-subtree of the left-subtree.



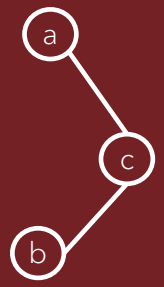
We shall now right-rotate the tree, making B the new root node of this subtree. C now becomes the right subtree of its own left subtree.



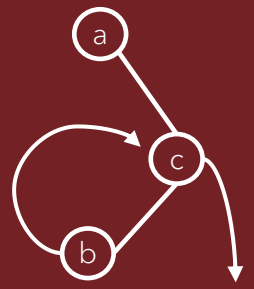
The tree is now balanced.

RIGHT-LEFT ROTATION

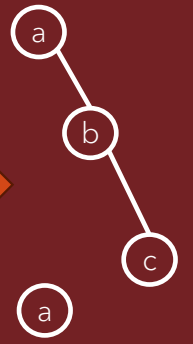
- It is a combination of right rotation followed by left rotation.



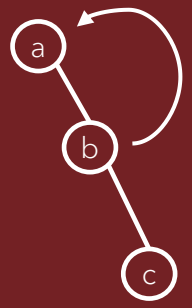
A node has been inserted into the left subtree of the right subtree. This makes A, an unbalanced node with balance factor 2.



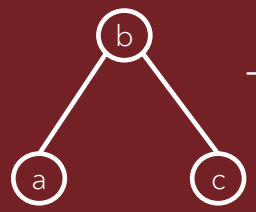
First, we perform the right rotation along C node, making C the right subtree of its own left subtree B. Now, B becomes the right subtree of A



Node A is still unbalanced because of the right subtree of its right subtree and requires a left rotation.



A left rotation is performed by making B the new root node of the subtree. A becomes the left subtree of its right subtree B.



The tree is now balanced.

THANK YOU

